Design Document

Project 1

Group: Mission Impossible
Jordan Amgai | Ved Ghimire | Gnaneswar Allu | Saksham Shrestha

Discussion / Introduction

The city has been dealing with a rise in traffic incidents and road accidents, and the city council wanted to do something about it. Their solution was to build a database that keeps track of every driver in the city including things like where they work, the areas they drive through most often, and whether they have received any traffic tickets. If a driver has a ticket, the system needs to store exactly when it was issued and where it happened. Having all of this in one place gives the city a much better picture of who is on the roads and where problems are most likely to come from.

The goal of this project is to build that driver database system in a way that is organized and efficient. Our program will start by reading in at least 100 driver records from a text file and loading them into the system. From there, the user will get a menu where they can: add new drivers, search for a specific driver by their ID, pull up the most recently licensed drivers, pull up the oldest licensed drivers, or move a driver to an inactive database if they leave the city. The system keeps two separate lists — one for active drivers and one for inactive ones — so records are never deleted.

System Requirements

- To store all the information about a driver, the program uses a base class called Driver that holds things like the driver's name, date of birth, address, license issue date, work category, years of experience, medical condition, and ticket details.
- Instead of storing things like dates and addresses as plain text strings, our program uses separate classes for them. A Data Class, an Address class, and a Ticket class which will be inside the Driver class (composition).
- On top of that, three classes called YouthDriver, MiddleAgeDriver, and SeniorDriver are built off the base Driver class using inheritance, since the city wants us to also classify drivers by age group (16-28 as youth, 28-50 as middle-aged, and 50+ as senior).
- For actually storing and managing all the driver records, our program uses 4 different data structures (built from scratch without using any STL of course).
- The linked list holds all the active driver records and makes it easy to insert drivers at the beginning, middle, or end of the list.
- Our hash table will work with the linked list and will let the program find any driver by their ID in O(1) time, meaning it won't have to scan through the whole list to find someone.
- We used a stack to track the newest licenses because they work as LIFO (last in first out), so popping off the top N items will give us the most recently added licenses.
- The queue we used does FIFO (first in first out), so it tracks the oldest licences, pulling from the front and giving access to the licences that have been in the system the longest.
- Lastly, a second linked list will handle inactive drivers separately.

Class Structure	Driver: Abstract base class with all common fields. YouthDriver, MiddleAgeDriver, SeniorDriver: Derived via inheritance. Composition classes: Date, Address, Ticket (nested inside Driver).
Data Structures	LinkedList: Holds active driver records; supports front, back, and mid insert. HashTable: O(1) driver lookup by ID using chaining with a linked list. Stack (LIFO): Tracks newly licensed drivers for newest-first retrieval. Queue (FIFO): Tracks oldest licensed drivers for oldest-first retrieval. Second LinkedList: Manages inactive driver records separately.
Key Constraints	Minimum 100 driver records loaded from a text file at startup. All data structures built from scratch — no STL containers. Records are never deleted; inactive drivers move to the inactive list.

Database Requirement

Insertion	Records can be inserted at the beginning, middle (by county match), or end of the active linked list.
Retrieve N Newest	Retrieve the N most recently licensed drivers using the Stack (LIFO). N is user-configurable at runtime.
Retrieve N Oldest	Retrieve the N oldest licensed drivers using the Queue (FIFO). N is user-configurable at runtime.
Inactive Database	Drivers who migrate out of the city are moved from the active linked list to a separate inactive linked list. Records are never deleted.
O(1) Search	A HashTable keyed on driverId provides O(1) average-case lookup to meet the council's fast-search requirement.

Design Components

This section details every class in the system, including its attributes, methods, and a brief description of its role.

Core Classes

Driver (Base Class)

The central entity of the system. Driver is an abstract base class that holds all fields shared by every driver in the database — personal info, professional details, medical conditions, license date, and any traffic ticket.

Driver
Attributes
`-- driverId : String`
`-- name : String`
`-- dob : Date`
`-- address : Address`
`-- workCategory : String`
`-- experienceYears : int`
`-- medicalConditions : String`
`-- licenseIssueDate : Date`
`-- ticket : Ticket`
Methods
`+ getDriverId() : String`
`+ getName() : String`
`+ getTicket() : Ticket`
`+ setTicket(t: Ticket) : void`
`+ displayInfo() : void {abstract}`

Subtypes of Driver class

Instead of creating classes for every category, we use enum to represent the types of driver.

Enum	Values
WorkCategory	STUDENT, GOVERNMENT, SELF_EMPLOYED, BUSINESS_OWNER, PRIVATE_SECTOR
ExperienceLevel	NEW (0–5 yrs), MODERATE (5–15 yrs), EXPERIENCED (15+ yrs)
MedicalCondition	FIT, VISION_IMPAIRED, UPPER_EXTREMITY, LOCOMOTOR_DISABILITY

Composition Classes

Instead of storing dates and addresses as plain strings, the system uses dedicated classes composed inside Driver. This improves validation, reusability, and readability.

Date

Stores a calendar date (day, month, year) and provides formatting, validation, and comparison utilities. Used for date of birth, license issue date, and ticket date.

Date
Attributes
-- day : int
-- month : int
-- year : int
Methods
+ toString() : String
+ isValid() : bool
+ compareTo(d: Date) : int

Ticket

Represents a traffic violation associated with a driver. Stores whether the driver has a ticket, along with the issue date and location. A driver with no ticket simply has hasTicket set to false.

Ticket
Attributes
-- hasTicket : bool
-- ticketDate : Date
-- location : String
Methods
+ issueTicket(d: Date, loc: String) : void
+ getLocation() : String
+ getDate() : Date

Address

Holds a structured mailing address for a driver. Separating address into its own class allows for equality comparison and clean string formatting without embedding logic in Driver.

Address
Attributes
-- street : String
-- city : String
-- county : String
Methods
+ toString() : String
+ equals(a: Address) : bool

Data Structure Classes

All data structures are implemented from scratch. No STL containers are used anywhere in the project.

DriverDatabase

The top-level controller class that owns all four data structures. It coordinates loading drivers from file, inserting them into the active list and hash table, moving them to inactive, and exposing retrieval operations.

DriverDatabase
Attributes
-- activeList : LinkedList
-- inactiveList : LinkedList
-- lookupTable : HashTable
-- recentStack : Stack
-- processQueue : Queue
Methods
+ loadFromFile(path: String) : void
+ addDriver(d: Driver) : void
+ searchDriver(id: String) : Node*
+ deactivateDriver(id: String) : bool
+ getNewestLicensed(n: int) : LinkedList
+ getOldestLicensed(n: int) : LinkedList
+ saveToFile(path: String) : void

LinkedList

A singly linked list used for both active and inactive driver collections. Supports insertion at the front or back, removal by ID, and sequential search. Two separate instances are maintained — one for active, one for inactive drivers.

LinkedList
Attributes
-- head : Node*
-- tail : Node*
-- size : int
Methods
+ insertFront(d: Driver) : void
+ insertBack(d: Driver) : void
+ remove(id: String) : bool
+ search(id: String) : Driver
+ getSize() : int
+ display() : void

Node

The building block of the linked list. Each node holds a reference to a Driver object and a pointer to the next node in the list.

Node
Attributes
-- data : Driver
-- next : Node*
Methods
+ getData() : Driver
+ getNext() : Node*

HashTable

Provides O(1) average-case lookup by driver ID. Uses a character-sum hash function and chaining (via an array of linked lists) to resolve collisions. The table size is fixed at 101 slots to minimize collisions while keeping memory use reasonable.

HashTable
Attributes
-- table : LinkedList[TABLE_SIZE]
-- TABLE_SIZE : int = 101
Methods
+ hash(id: String) : int
+ insert(d: Driver) : void
+ search(id: String) : Driver
+ remove(id: String) : bool
+ display() : void

Stack

A LIFO (Last In, First Out) structure used to track drivers by license issue order. Pushing adds a newly licensed driver to the top; popping retrieves the N most recently licensed drivers.

Stack
Attributes
-- top : Node*
-- size : int
Methods
+ push(d: Driver) : void
+ pop() : Driver
+ peek() : Driver
+ isEmpty() : bool

Queue

A FIFO (First In, First Out) structure used to track the oldest drivers in the system. Drivers are enqueued when added, and dequeued from the front to retrieve those with the earliest license dates.

Queue
Attributes
`-- front : Node*`
`-- rear : Node*`
`-- size : int`
Methods
`+ enqueue(d: Driver) : void`
`+ dequeue() : Driver`
`+ peek() : Driver`
`+ isEmpty() : bool`

Relationships

The table below summarizes all class relationships in the system. Every relationship is a has-a composition, meaning the containing object owns and manages the lifetime of the contained object.

From	Relationship	To	Type
Driver	has-a	Date	Composition
Driver	has-a	Address	Composition
Driver	has-a	DOB	Composition
Node	has-a	Driver (data)	Composition
LinkedList	has-a	Node (head, tail)	Composition
DriverDatabase	has-a	LinkedList (active)	Composition
DriverDatabase	has-a	LinkedList (inactive)	Composition
DriverDatabase	has-a	Stack	Composition
DriverDatabase	has-a	Queue	Composition

Algorithms

The pseudocode below describes the core operations performed by the system.

Add Driver

```
Algorithm AddDriver(d)
  Insert driver into activeList
  Insert driver into lookupTable
  Push driver onto recentStack
  Enqueue driver into processQueue
```

Search Driver

```
Algorithm SearchDriver(id)
  index <- hash(id)
  Go to lookupTable[index]
  Traverse linked list in that bucket
  IF driver with matching id found
    RETURN driver
  ELSE
    RETURN null
```

Deactivate Driver

```
Algorithm DeactivateDriver(id)
  driver <- SearchDriver(id)
  IF driver = null
    RETURN false
  Remove driver from activeList
  Remove driver from lookupTable
  Insert driver into inactiveList
  RETURN true
```

Retrieve N Newest Licensed Drivers

```
Algorithm GetNewestLicensed(n)
  Create empty list result
  count <- 0
  WHILE stack not empty AND count < n
    driver <- pop recentStack
    Add driver to result
    count <- count + 1
  END WHILE
  RETURN result
```

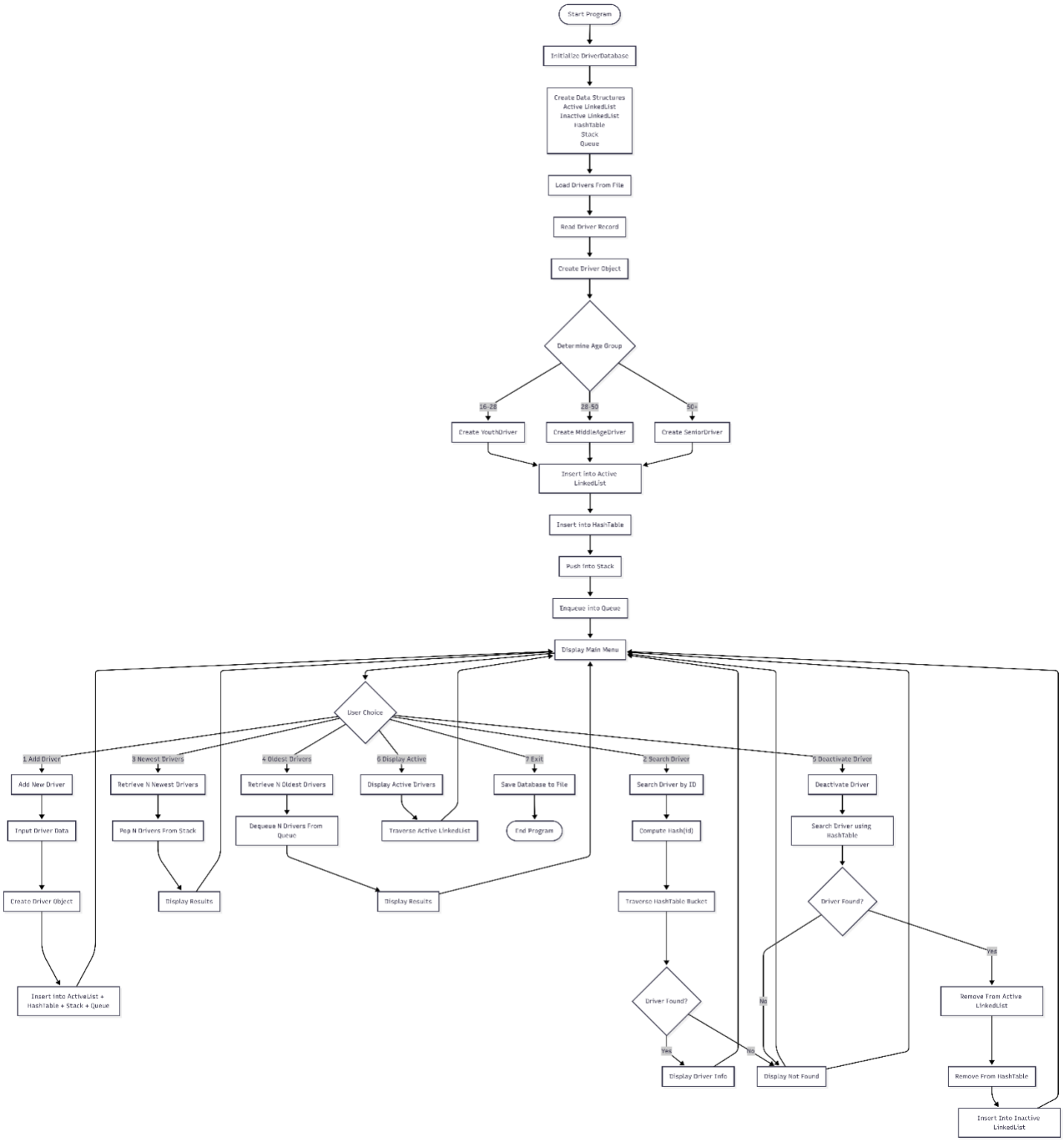
Retrieve N Oldest Licensed Drivers

```
Algorithm GetOldestLicensed(n)
  Create empty list result
  count <- 0
  WHILE queue not empty AND count < n
    driver <- dequeue processQueue
    Add driver to result
    count <- count + 1
  END WHILE
  RETURN result
```

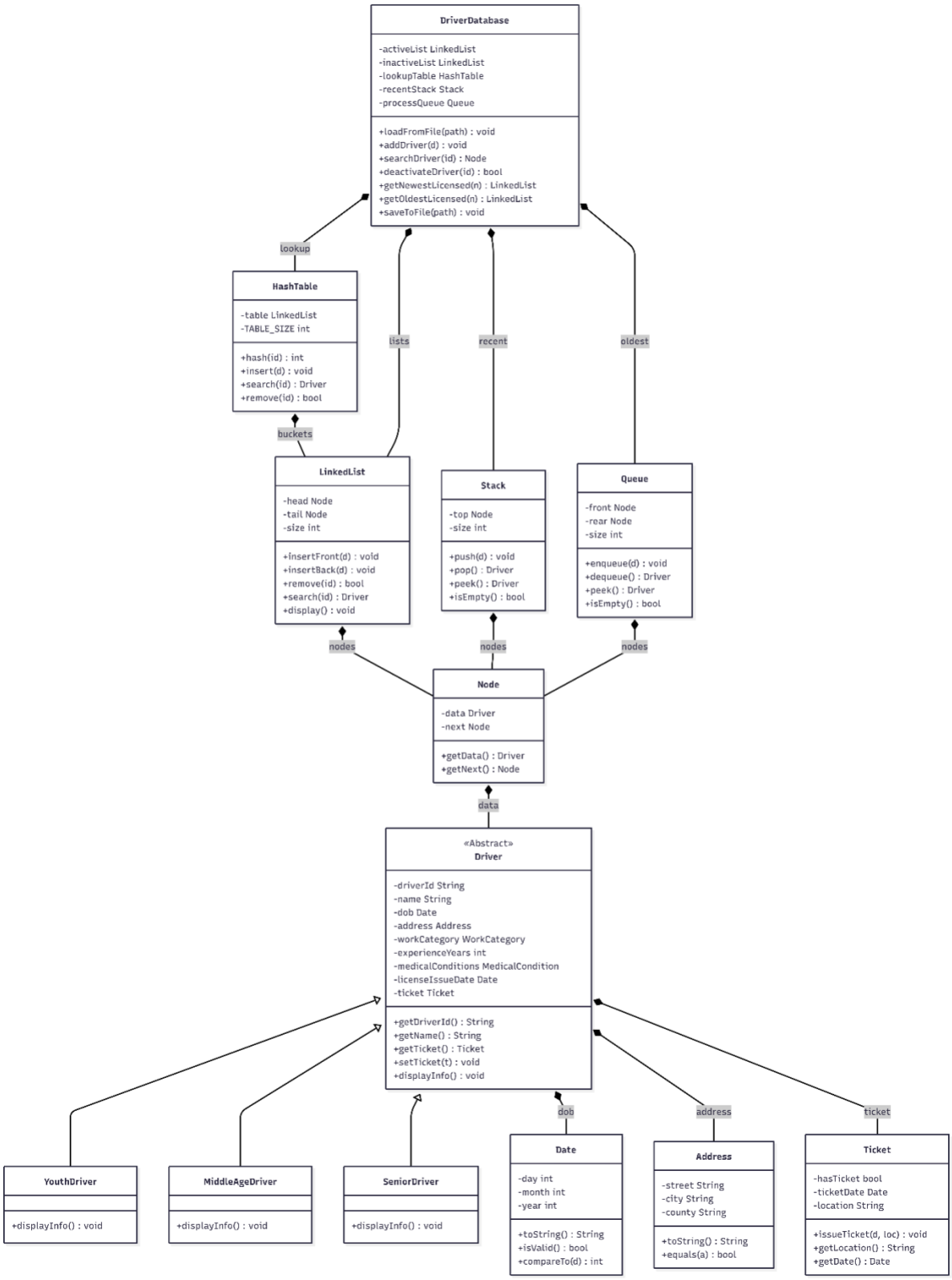
Hash Function

```
Algorithm Hash(id)
  sum <- 0
  FOR each character c in id
    sum <- sum + ASCII(c)
  END FOR
  index <- sum mod TABLE_SIZE
  RETURN index
```

FLOWCHART


Start Program
Initialize DriverDatabase
Create Data Structures
Active LinkedList
Inactive LinkedList
HashTable
Stack
Queue
Load Drivers From File
Read Driver Record
Create Driver Object
Determine Age Group
16-28
28-50
50+
Create YouthDriver
Create MiddleAgeDriver
Create SeniorDriver
Insert into Active LinkedList
Insert into HashTable
Push into Stack
Enqueue into Queue
Display Main Menu
User Choice
1 Add Driver
3 Newest Drivers
4 Oldest Drivers
5 Display Active
7 Exit
2 Search Driver
9 Deactivate Driver
Add New Driver
Retrieve N Newest Drivers
Retrieve N Oldest Drivers
Display Active Drivers
Save Database to File
Search Driver by ID
Deactivate Driver
Input Driver Data
Pop N Drivers From Stack
Dequeue N Drivers From Queue
Traverse Active LinkedList
End Program
Compute Hash(Id)
Search Driver using HashTable
Create Driver Object
Display Results
Display Results
Traverse HashTable Bucket
Driver Found?
Insert into ActiveList + HashTable + Stack + Queue
Driver Found?
Yes
No
Yes
No
Remove From Active LinkedList
Display Driver Info
Display Not Found
Remove From HashTable
Insert Into Inactive LinkedList

UML


DriverDatabase
-activeList LinkedList
-inactiveList LinkedList
-lookupTable HashTable
-recentStack Stack
-processQueue Queue
+loadFromFile(path) : void
+addDriver(d) : void
+searchDriver(id) : Node
+deactivateDriver(id) : bool
+getNewestLicensed(n) : LinkedList
+getOldestLicensed(n) : LinkedList
+saveToFile(path) : void
lookup
lists
recent
oldest
HashTable
-table LinkedList
-TABLE_SIZE int
+hash(id) : int
+insert(d) : void
+search(id) : Driver
+remove(id) : bool
buckets
LinkedList
-head Node
-tail Node
-size int
+insertFront(d) : void
+insertBack(d) : void
+remove(id) : bool
+search(id) : Driver
+display() : void
Stack
-top Node
-size int
+push(d) : void
+pop() : Driver
+peek() : Driver
+isEmpty() : bool
Queue
-front Node
-rear Node
-size int
+enqueue(d) : void
+dequeue() : Driver
+peek() : Driver
+isEmpty() : bool
nodes
nodes
nodes
Node
-data Driver
-next Node
+getData() : Driver
+getNext() : Node
data
«Abstract»
Driver
-driverId String
-name String
-dob Date
-address Address
-workCategory WorkCategory
-experienceYears int
-medicalConditions MedicalCondition
-licenseIssueDate Date
-ticket Ticket
+getDriverId() : String
+getName() : String
+getTicket() : Ticket
+setTicket(t) : void
+displayInfo() : void
dob
address
ticket
YouthDriver
+displayInfo() : void
MiddleAgeDriver
+displayInfo() : void
SeniorDriver
+displayInfo() : void
Date
-day int
-month int
-year int
+toString() : String
+isValid() : bool
+compareTo(d) : int
Address
-street String
-city String
-county String
+toString() : String
+equals(a) : bool
Ticket
-hasTicket bool
-ticketDate Date
-location String
+issueTicket(d, loc) : void
+getLocation() : String
+getDate() : Date